

AI Video Ad Generation Pipeline

Final Project Report — ad-pipeline-independent

Prepared: August 27, 2026

Prepared by: Yulin (Dexter) Peng

Prepared for: Internship final submission / technical review

Repository: github.com/DexterPeng117/DigitalMarketingAgent

1. Executive Summary

ad-pipeline-independent is a five-stage, clean-room pipeline that turns raw product photographs into a finished, narrated, subtitled video advertisement, built solo over roughly four weeks (August 1–August 27, 2026). The pipeline chains real, paid third-party AI services — an LLM for storyboarding, an image model for scene compositing, a video model for animation, and a text-to-speech model for narration — behind five independent Python scripts (2,070 lines total) with no shared code from any existing pipeline.

The system has been exercised end-to-end against real product photos with real API calls, is covered by a 68-test automated pytest suite (all passing), is MIT-licensed, and carries a clean, single-author git history with no AI co-authorship trailers. Two components remain incomplete by design: a local GPU rendering backend (implemented but untested, no GPU access available) and automatic background-music generation (blocked by a provider-side regional restriction, with a manual override left in place).

2. Project Objective & Scope

The objective was to build, independently and from scratch, a working pipeline that automates one of the more labor-intensive parts of digital marketing production: turning static product photography into a short-form video ad ready for social platforms. “Clean-room” was a hard constraint — no source code was copied or referenced from any existing pipeline, including a more mature reference pipeline built by a colleague on the same team, which was used only as a visual quality benchmark, never as source material.

Scope covers the full path from raw photo to a tracked, published asset:

- Sorting raw product photos into named views (front / side / back).
- Generating an LLM-authored storyboard and narration script from those views.
- Compositing the product into AI-generated marketing scenes.
- Animating those scenes into video.
- Adding narration, subtitles, and (optionally) music.
- Registering and tracking the finished ad's publication and performance.

3. System Architecture Overview

The pipeline is five sequential scripts, each independently runnable, chained by one orchestration shell script. Each stage reads the previous stage's output from disk — there is no shared in-memory state — which keeps every stage independently testable and re-runnable.

Stage	Script	Input → Output	Key technology
1	extract_product_views.py	Raw product photo(s) → named view PNGs (assets/<brand>/<view>.png)	Vision-capable LLM classification + grid-split fallback
2	ad_director.py	Product views (+ optional brief) → storyboard spec JSON + AI-composited scene images	LLM storyboard generation; Qwen Image 3 Pro scene compositing
3	render_pipeline.py	Storyboard spec → silent rendered video	Wan 2.7 (cloud) / Wan 2.2 (local ComfyUI) / free interpolation backend
4	finalize_ad.py	Silent video → finished ad (narration + subtitles + music)	TTS synthesis; ffmpeg mux; soft mov_text subtitles
5	ad_tracker.py	Finished ad → tracked record	CSV/XLSX-backed publish + performance tracking

3.1 Workflow

1. A raw product photo is classified into front / side / back views.
2. An LLM turns those views (plus an optional creative brief) into a storyboard spec: shot list, tagline, narration script, and a per-shot scene prompt.
3. Each shot's scene prompt is sent to an image model, which composites the actual product photo into a generated marketing scene.
4. The rendering backend (selected per-spec) animates the scene sequence into a silent video.
5. Narration is synthesized from the script, subtitles are generated by evenly splitting the script across the narration's measured duration, and both are muxed with the video.
6. The finished ad is registered in the tracker; publish and performance data can be recorded per platform afterward.

4. Model & API Selection

All model access runs through a single OpenRouter account across four distinct endpoint families — chat completions, an independent image-generation endpoint, a dedicated video-generation endpoint (submit / poll / download), and a TTS endpoint. Model choice per stage was constrained less by raw capability and more by which endpoints actually supported the required input modality and which provider/data-policy combinations were unblocked on the account (see §6).

Stage	Model / service	Why
Storyboard + narration script	Vision-capable OpenRouter chat model (e.g. Gemma, DeepSeek for text-only steps)	Must accept image input (product views) directly in the prompt; free/low-cost tiers kept iteration cheap during account-restriction troubleshooting
Scene compositing	Qwen Image 3 Pro (Alibaba)	Only model on the account able to composite a real product photo into a generated scene at production quality, via OpenRouter's independent images endpoint
Video generation (cloud)	Wan 2.7 (Alibaba)	OpenRouter's dedicated video endpoint (submit/poll/download); real cost ≈\$0.50 per generation; resilient polling added after early connection-hang failures

Stage	Model / service	Why
Video generation (local, optional)	Wan 2.2 FLF2V via self-hosted ComfyUI	Zero marginal cost alternative backend; fully implemented and wired in without touching the cloud backend, but untested end-to-end (no GPU environment available)
Narration (TTS)	fish-audio/s2.1-pro-free:free	Only \$0/M-character option on OpenRouter's TTS collection; explicitly a prototyping-tier choice, swappable via config for production volume
Background music	Google Lyria 3 (blocked)	Only music-generation model available on OpenRouter; returns a region restriction on this account — left as a manual --music override rather than blocking the pipeline

5. Development Workflow & Tooling

The implementation was carried out with Claude (via the Claude Code CLI) as the coding agent, under a deliberately strict verification discipline rather than an autonomous one: every implementation step was specified as a detailed written prompt, the agent's resulting code and test output were independently re-verified against the actual repository — reading diffs, re-running the test suite, probing real API responses, and inspecting rendered video with ffprobe — before being accepted or built upon. The agent's own summary of what it did was never taken as ground truth.

This mattered in practice: several agent-reported “fixes” (an OpenRouter account-restriction workaround, a claimed commit that turned out not to exist on any branch) did not hold up under independent verification and were caught before being relied on.

A standing project rule was to keep every commit single-authored: no Co-Authored-By trailers naming the AI assistant. This required two remediation passes over the project's history — a git filter-branch rewrite, and eventually a migration to a fresh repository after GitHub's contributor-graph cache kept surfacing the assistant as a contributor despite clean underlying commit data.

6. Testing & Quality Assurance

Automated coverage focuses on the pure-logic paths of each script — CSV/XLSX tracker state, spec parsing and validation, sentence-splitting for subtitles, backend dispatch — with all network and subprocess calls mocked via pytest's monkeypatch, tmp_path, and capsys fixtures.

Test file	Covers	Tests
test_ad_tracker.py	register / publish / metrics / report / export / scan commands	23
test_ad_director.py	Storyboard + scene-generation logic	19
test_render_pipeline.py	Backend dispatch and rendering logic	12
test_finalize_ad.py	TTS, subtitle generation, audio/video muxing logic	14

Total: 68 tests, all passing (python -m pytest tests/ -q → 68 passed in 1.25s, independently re-run and confirmed).

Beyond unit tests, the pipeline was exercised manually end-to-end with real, paid API calls. Output videos were independently verified with ffprobe rather than taken on report alone — e.g. a representative finished ad measured 11.5s at 1080×1920 with synchronized video, audio, and 3 subtitle cues, narration covering ≈10.8s of the 11.5s runtime.

7. Cost Analysis

Real, observed per-ad costs on the current model selection:

Category	Cost per finished ad	Notes
Video generation (Wan 2.7, cloud)	≈\$0.50 per clip generated	Dominant cost; 1–2 clips per ad depending on shot count; free local backend available but untested
Scene image compositing (Qwen Image 3 Pro)	Low, per-image	2–3 scene images per ad; exact per-image rate not separately isolated from account billing
Storyboard + narration LLM	\$0 (free tier)	Vision-capable free-tier model used throughout
TTS narration	\$0 (free tier)	fish-audio free tier — explicitly a prototyping allowance, not rate-limited for production volume

At current model selection, cloud video generation is the only meaningfully metered cost per ad; every other stage runs on a free tier. Scaling to production volume would require budgeting for a paid TTS tier (the free tier is not intended for volume use) and re-evaluating whether the free-tier LLM and image models hold up under higher throughput and rate limits.

8. Known Gaps & Limitations

- `wan_flf_local` (the self-hosted ComfyUI + Wan 2.2 backend) is fully implemented and wired into the backend dispatch table, but has not been tested end-to-end — no GPU/ComfyUI environment has been available during development.
- Automatic background-music generation is not implemented. OpenRouter's only music-generation model (Google Lyria 3) returns a region restriction on this account; `finalize_ad.py --music` still accepts a manually supplied track as a workaround.
- Subtitle timing is an even split by sentence count across the narration's measured duration, not true word-level audio alignment — the TTS endpoint returns raw audio bytes with no timing metadata.
- Subtitles are soft-mounted (MP4 `mov_text`) rather than burned into the picture, because the available ffmpeg build lacks `libass/libfreetype`; some short-form platforms re-encode uploads and drop soft subtitle tracks.
- The three-view product photo approach (classifying a raw photo into front / side / back before storyboarding) still has room for optimization. This was raised in discussion with a colleague (William), who suggested an alternative approach; that alternative is not yet mature enough to adopt and has not been implemented, but is a candidate for a more complete redesign of this stage in a future iteration.

9. Repository, Documentation & Governance

- Public repository: github.com/DexterPeng117/DigitalMarketingAgent — description, topics (ai-video-generation, llm, advertising, python, openrouter), and a sample-output section (scene composite + finished-ad frame) added to the README.
- MIT LICENSE added; single contributor confirmed on GitHub (no AI co-authorship in commit history, verified after a full-history grep and, ultimately, a clean repository migration).
- README documents directory structure, the five-stage pipeline, current implementation status, and known gaps rather than overstating completeness.

- The project is also featured on the author's personal GitHub profile README alongside an existing published project.

10. Cross-Team Collaboration: product-ad-pipeline (Team Repository)

Separate from the independent build, a component was contributed to a teammate's (William Kang) more mature reference pipeline, product-ad-pipeline, in the team's shared repository. This work is called out separately from §§1–9 because it followed a different process — collaborative, on someone else's codebase — rather than the solo, clean-room build described above.

10.1 Contribution: ad_tracker.py

An MVP publishing/analytics tracker (ad_tracker.py) was built and tested end-to-end against a real generated spec (watch_wan_flf.json) before being handed off for review. Because the team's contribution workflow wasn't yet familiar, the first handoff was by email — script, a short operational manual (setup, commands, test results), and an explicit question about whether PR was the right channel. The reviewer confirmed the approach fit a real gap (finalize_ad.py produced the video but nothing recorded publish state), approved a PR, requested standard housekeeping (exclude outputs/ CSVs and placeholder media via .gitignore, add a usage note to the README, add missing dependencies to requirements.txt), and set GitHub PRs as the preferred channel going forward. A pull request adding ad_tracker.py (PR #11) was opened and approved, and is ready to merge.

10.2 Unblocking a Shared Dependency

While bringing up the rest of the team pipeline locally (extract_views → ad_director → render → finalize_ad), a startup failure was traced to product_consistent_ad_pipeline.py unconditionally importing an external story_reel module that lived outside the repository, on the maintainer's own machine — confirmed by reading the run() function directly rather than assuming, since the maintainer's initial “already fixed” report did not match the code on main when re-pulled. Rather than wait on the upstream fix, a minimal local stub was built implementing only the interface the interp and wan_flf backends actually needed (the ltx-only functions were left as explicit NotImplementedError, since that path wasn't in use). With the stub in place, the full pipeline was run end-to-end twice: first on the zero-cost interp backend to validate correctness, then on wan_flf for production-quality output, with English narration under a luxury_british voice persona. The maintainer subsequently confirmed the story_reel/ComfyUI dependency had been removed from main for the interp and wan_flf backends, making the stub unnecessary going forward except as a reference for older checkouts.

10.3 Creative Feedback Loop

Review of the resulting ad surfaced a genuine design flaw: the three product views (front/side/back) were being driven as a forced shot sequence — scene 1 front, scene 2 side, scene 3 back — which reads as a product turntable rather than an advertisement. The maintainer treated this as a director-level fix: views should function as an identity reference, not a mandatory camera order, defaulting every beat to the product's preferred view and changing view only when a beat's story genuinely calls for it.

10.4 Open Item

The maintainer referenced a specific fix commit (c1b1f00) implementing the view-handling change above. That commit could not be located after a full search — latest main pulled, every remote branch fetched (including all william/* branches), an org-wide GitHub commit search, and a check for forks (none exist) — and remains flagged back to the maintainer for clarification as of this report.

11. Lessons Learned

- Multimodal input constraints surfaced as opaque 404s (“no endpoint available”) rather than clear validation errors when a text-only model was sent an image-bearing prompt — costly to diagnose without the underlying capability being stated up front.
- Account-level provider allowlists and data-policy toggles (not model capability) were the actual blocker behind most “model doesn't work” symptoms during setup — diagnosing this required probing raw API responses rather than trusting error messages at face value.
- Environment constraints shape design decisions as much as feature requirements do: the choice between burned-in and soft subtitles was decided by which ffmpeg build was actually available, not by which approach was theoretically better.
- Benchmarking output quality against a colleague's more mature reference pipeline (without referencing its code) was an effective way to close a visible quality gap in scene generation.
- Independently re-verifying an AI coding agent's self-reported results — rather than trusting its summary — caught at least two cases where the reported outcome did not match the actual repository state.
- On a shared team codebase, verifying a maintainer's status report against the actual code on main (rather than assuming a reported fix had landed) caught a real discrepancy early and kept a blocked task moving via a scoped local workaround.

12. Conclusion / Final Recommendation

The pipeline is functionally complete across all five stages, exercised against real paid API calls, covered by an automated test suite, documented, MIT-licensed, and carries an unambiguous single-author history — ready as an internship deliverable in its current state.

Three concrete next steps would close the remaining gaps: (1) validate the local ComfyUI/Wan 2.2 rendering backend once GPU access is available, to provide a genuinely free rendering path; (2) resolve or route around the regional restriction blocking automatic background-music generation, either through an alternate model or an approved account change; and (3) revisit the three-view product photo approach with a more complete design, once the alternative discussed with William is mature enough to adopt.

On the collaborative side, two items remain open: merging the approved `ad_tracker.py` PR (#11) into the team repository, and getting clarification from the maintainer on the location of fix commit `c1b1f00`.

Appendix: Technology Stack Summary

Layer	Tool / service	Notes
View classification	Vision-capable OpenRouter chat model	Grid-split fallback for ambiguous photos
Storyboard generation	OpenRouter chat/completions (vision-capable, free tier)	Produces shot list, tagline, narration script, scene prompts
Scene compositing	Qwen Image 3 Pro (Alibaba)	OpenRouter independent <code>/api/v1/images</code> endpoint
Video generation (cloud)	Wan 2.7 (Alibaba)	OpenRouter <code>/api/v1/videos</code> ; <code>submit/poll/download</code> ; ≈\$0.50/clip
Video generation (local)	Wan 2.2 FLF2V via ComfyUI	Implemented, untested — no GPU environment
Narration	fish-audio/s2.1-pro-free:free	OpenRouter <code>/api/v1/audio/speech</code>

Layer	Tool / service	Notes
Subtitles	ffmpeg mov_text soft track	Even split by sentence over measured narration duration
Muxing	ffmpeg	Stream-copied video, AAC audio, no re-encode
Publish tracking	ad_tracker.py + CSV/XLSX	register / publish / metrics / report / export / scan
Testing	pytest	68 tests across 4 files, all passing
License	MIT	Added 2026-08-27
Version control	git / GitHub	Single-author history, no AI co-authorship trailers